

Report I

Simple numerical operations with vectors and matrices

Author: Patryk Kuna

1. Introduction

Linear algebra operations such as scalar products, matrix–vector multiplications and matrix-matrix multiplication are fundamental in numerical computing. They appear in many scientific applications including simulations, optimization methods and iterative algorithms used in physics. Efficient implementation of these operations is both crucial and beneficial for high-performance numerical calculations.

The scalar product of two vectors $\vec{x}$ and $\vec{y}$ of length n is defined as

$$s = \vec{x} \cdot \vec{y} = \sum_{i=0}^{n-1} x_i y_i$$

it can be used to compute the Euclidean norm (length) of a vector

$$\|\vec{x}\| = \sqrt{\vec{x}^T \vec{x}}$$

Matrix-vector multiplication is defined as

$$y_i = \sum_{j=0}^{n-1} A_{i,j} x_j$$

where A is an $n \times n$ matrix and $\vec{x}$ is a vector. This operation is widely used in iterative algorithms, where a vector is repeatedly transformed by multiplication with a matrix.

Finally, a matrix-matrix multiplication is defined as

$$C_{i,k} = \sum_{j=0}^{n-1} A_{i,j} B_{j,k}$$

This operation is computationally expensive because its complexity grows as $O(n^3)$. For large matrices it is therefore common to use optimized numerical libraries such as BLAS - it provides highly optimized routines for vector and matrix operations that

take advantage of hardware-level optimizations such as cache usage and parallelization.

In this project, vector and matrix operations were implemented in C++ language and compared with optimized implementations from the BLAS3 library.

2. Description of the Numerical Task

The goal of this project was to implement basic numerical operations on vectors and matrices stored in a one-dimensional array using row-major ordering. The matrix size was set to

$$n = 2000$$

which results in a matrix containing $2000 \cdot 2000 = 4 \cdot 10^6$ elements.

The matrix elements were defined using the formula

$$A_{i,j} = \frac{1}{|i - j|^4 + \gamma}$$

where $i, j = 0, 1, \dots, n - 1$ and the parameter gamma γ takes values

$$\gamma = 2 \text{ and } \gamma = 3$$

The numerical task consisted of three main parts:

1. Iterative matrix-vector multiplication

The vector $\vec{x}$ was initially filled with ones

$$x_i = 1$$

and the multiplication

$$\vec{x}^{(m)} = A^m \vec{x}$$

was performed iteratively for $m = 30$ steps. After each multiplication the Euklidean norm of the resulting vector was computed and written to a text file.

2. Matrix-matrix multiplication

The matrix product

$$C = A \cdot A$$

was calculated using nested loops corresponding to the mathematical definition of a matrix multiplication.

3. Matrix-matrix multiplication using BLAS library

The same operation was computed using the BLAS procedure `cblas_dgemm`. Execution times of both implementations were measured using C++ `time.h` library using the `difftime` function in order to compare its performance.

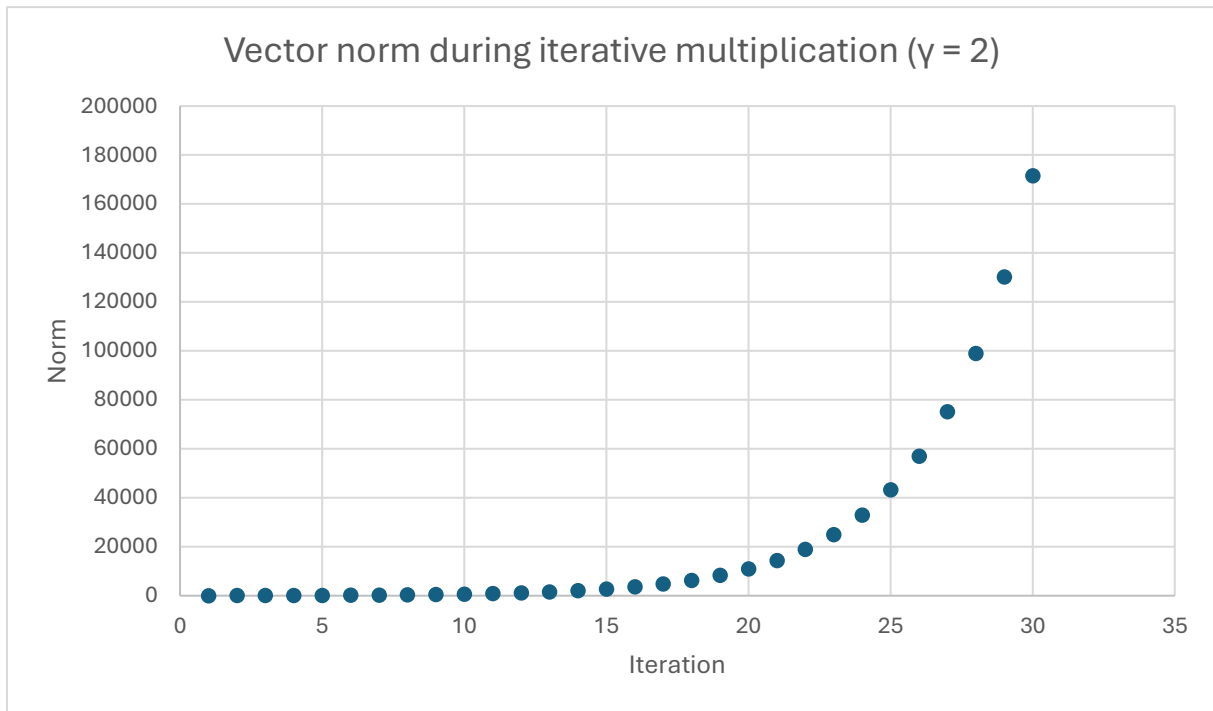
3. Results

The results consist of two parts: analysis of the vector norm during iterative multiplication and comparison of execution times for matrix multiplication.

3.1 Norm evolution during matrix-vector multiplication

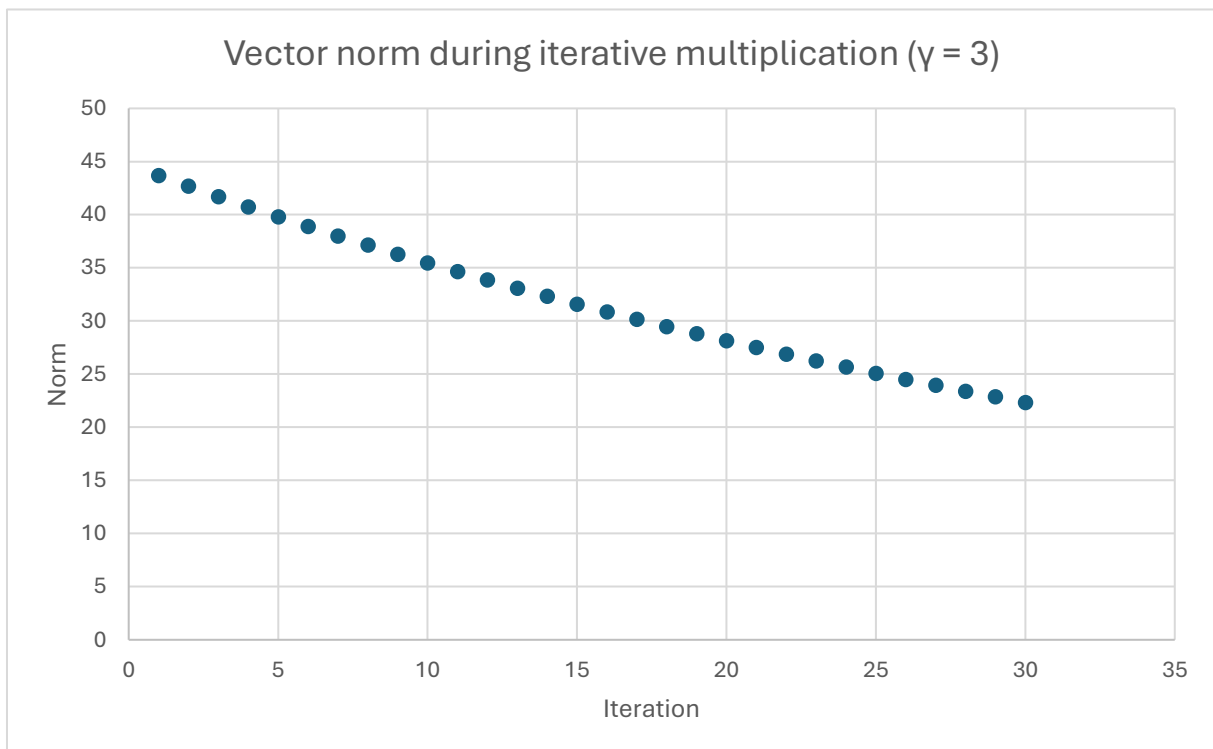
The Euclidean norm of the vector was calculated after each iteration of the multiplication $x^{(i)} = Ax^{(i-1)}$. The results for both values of the parameter γ are shown in Figures 1 and 2.

Figure 1. Norm of the vector for $\gamma = 2$ during 30 iterations.



The norm increases rapidly with each iteration. This indicates that the matrix acts as an amplifying transformation for this value of γ .

Figure 2. Norm of the vector for $\gamma = 3$ during 30 iterations.



For $\gamma = 3$, the vector norm decreased with each iteration. This means that the transformation reduces the magnitude of the vector and the sequence converges toward smaller and smaller values.

These results confirm the expected behavior of the system for the two different parameters.

3.2 Performance of matrix-matrix multiplication

Execution times of the two implementations were measured:

Performance for $\gamma = 2$

Method used	Execution time
Iterative approach	161 s
BLAS3 dgemm	1 s

Performance for $\gamma = 3$

Method used	Execution time
Iterative approach	173 s
BLAS3 dgemm	less than 1 s

The BLAS implementation was significantly faster than the manually implemented, iterative algorithm. This was expected because of BLAS routines which are highly optimized and are using very efficient memory access patterns and vectorization.

4. Conclusions

This project demonstrated the implementation of fundamental numerical operations used in scientific computing. Matrix and vector operations were implemented in C++ using one-dimensional arrays with row-major storage.

The iterative matrix-vector multiplication showed different behaviors depending on the parameter γ . For $\gamma = 2$, the norm of the vector increased rapidly with each iteration, while for $\gamma = 3$ the norm decreased, indicating a contracting transformation.

The comparison between manual matrix multiplication and the BLAS routine `dgemm` showed the importance of optimized numerical libraries. The BLAS implementation was significantly faster due to the low-level optimizations and better memory usage.